

BuildStorm 调优文档（初版）

第 1 章 概述

1.1 背景概述

在初赛过程中我们渐渐发现:rCore 的进程线程模型(PCB+TCB)等内容以教学意义为主,实际不能很好地适配 linux 语义与多核并行支持的需要。因此我们先重构了进程线程框架,然后才开始针对 BuildStorm 的特定方面做优化。

1.2 优化内容概况

相较于初赛阶段,我们的内核新增/优化了以下特性:

1. 完善的多核并行处理支持,通过锁与原子变量结合各类边界情况约束,保证多核并行的正确性。
2. 具备 COW 语义的 `exec` 按需加载与代码段零拷贝,支持懒分配与完整的 `page fault` 处理。
3. 地址空间翻译的 `pin` 机制,以及内存管理模型的初步 RCU 语义。
4. `futex`、`epoll`、`ppoll` 基于等待队列实现真正阻塞,替代了初赛阶段“睡眠轮询”的过渡语义。

5. 内核工作线程：将磁盘回写、软件计时器 `tick`、网络 `poll` 等周期性工作交由独立的内核工作线程执行，不再占用普通用户进程的 `trap` 流程。
6. 合并页缓存与块缓存管理，减少磁盘访问的一次拷贝；用更快的近似 LRU 替换原有 LRU 算法。
7. 彻底重构 `pcb + tcb` 的进程线程模型：参考 Linux 的线程模型，删除 `pcb` 包装，任务通过线程组组织为进程，跨线程共享的资源以 Arc 管理，并在 `clone` 时依据 `flags` 参数决定保留的资源与新旧任务关系。
8. 完整的类 Linux 多队列调度，含 CFS、FIFO 等调度类型，并实现基于“主动任务窃取”和“任务唤醒时优先选择空闲核 + IPI 休眠唤醒”的负载均衡模型。
9. 基于核间 IPI 的 TLB 跨核刷新与按需刷新。RISC-V 实现用户态共享内核页表：将内核映射至高半地址空间，用户页表除映射用户页外，还通过复制内核高半根页表引用内核页表，以减少 `satp` 切换及相应的 TLB 刷新开销。
10. 细化各类锁的粒度，实现写者优先（区别于 `spin` 库的写者不公平策略）的读写锁 `RwLock`，并广泛用于页缓存、内存管理结构等读多写少路径，替换原有的 `Mutex`。
11. 完成 `ext4fs` 的多线程并发安全适配与元数据校验码更新。

第2章 重构与多核适配

2.1 问题概述

实际上初赛阶段有已经具备了多核 SMP 的早期框架，但后来为了跑测试，分数压力有点大，于是便淡出了相关内容的调试。为了支持决赛的 buildstorm，我们必须修改框架，重新支持多核。期间遇到了许多问题，这里以两个典型为例。

2.2 ABBA 锁问题

2.2.1 问题表现：偶发卡死

2.2.2 调试过程：（以 lmbench 举例）

1. 根目录键入 `make debug-rv INIT=sh RV_SMP=8`
2. 新开一个终端键入 `make gdb-rv` （此处使用的是 vscode）
3. debug 终端中我们的内核会进入 `busybox sh`（初赛的 shell），运行一个用户程序，如 `lmbench`，等待卡死（运行较长时间不退出），若偶然没卡死就再运行一遍
4. 切换到 gdb 终端（集成开发环境），键入 `ctrl+c` 暂停，观察调用堆栈如下图

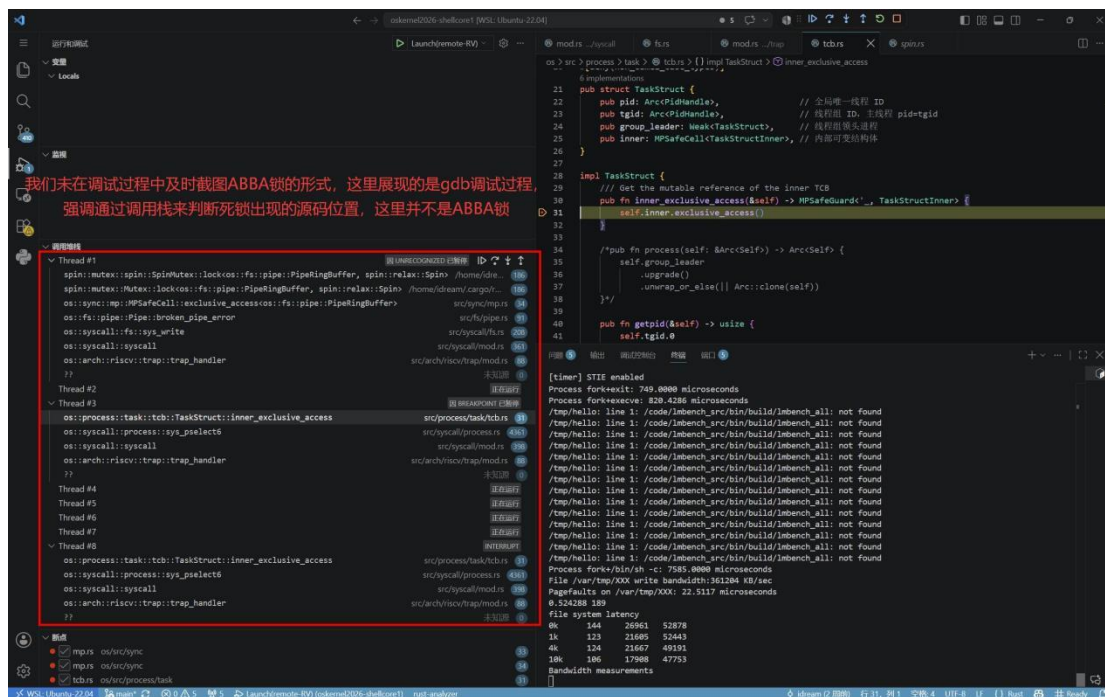


图1 vscode + gdb 调试界面

5. （开发过程中没有专门截图，上方图片与本问题不直接相关，仅用于演示 gdb 的调用栈等的可视化）红框中有 8 个 thread，对应 8 个核。Ctrl+C 后，如上图，如果显示正在运行但未显示调用栈是表示核在运行用户态指令。由于死锁肯定是我们写的内核的问题。如果不确定哪几个核死锁可以再运行一会儿，最后看出哪两个核的调用堆栈很久没变，再观看调用堆栈上的函数以及其源码。

2.2.3 更改过程

把锁序调整一下即可，ABBA 死锁改为 ABAB 形式，即对于某个特别范式，两个进程都先拿锁 A，再拿锁 B 即可不成环

2.3 唤醒进程时，其可能在保存好上下文之前被调度

2.3.1 问题表现

开启多核后偶发的任务无输出，或者是无法处理的用户态缺页 `user_page_fault`。问题出现在 `futex` 阻塞恢复相关。

2.3.2 调试过程

首先我们通过打印逻辑段，发现访址位置在用户态页表逻辑段之外，亦或者是根本没有输出。检测 `mmap` 和 `munmap` 以及 `brk` 都是合法操作。而这种上下文在用户态页表逻辑段之外的用户态缺页或者根本没有输出的情况大概率是与上下文值不同有关（这是在调单核系统调用时出现的现象）。开启 `warn` 并再次遇到此问题后对比上下文发现上下文确实不同。

通过反复的执行发现，以及控制变量（开多核会出现问题，开单核不会），我们把问题定位在了 `futex` 在多核下的环境。多核环境下，两个线程进行 `futex_wait` 和 `futex_wake` 可以在同时执行（或者其它的阻塞，恢复逻辑）。

最终发现，由于我们的线程是先入队再保存上下文，而在入队与保存上下文间，如果线程被唤醒调度，则会拿

到不完整的上下文。

2.3.3 更改过程

尝试过一些解决问题的方法：

1. 先保存上下文再入队。但入队本身的操作会推进上下文，所以这个方案是错误的。

2. 使用一个锁让这段操作原子性。先入队后保存上下文然后切换到调度器流程释放锁。但由于调度过程会切换到其它执行流，这里的锁生命周期难以界定，因此没有采纳。

3. 新定义了一个 BLOCK_SAVING 状态，即“阻塞保存上下文中”状态。在保存好上下文切换到调度器后，调度器将 BLOCK_SAVING 改成 BLOCKED 态，而其他核的唤醒函数如果遇到 BLOCK_SAVIN 态的任务则原地自旋等待。最终选择了这个方法。

第3章 BuildStorm 针对性调优

3.1 首次完整运行

我们在支持了基础的系统调用后，2.1 和 2.2 可以跑通，

2.3 编译花费了 105 分钟，十分漫长：

```
Compiling ax-api v0.7.4 (/work/tgskits/os/arceos/api/arceos_api)
Compiling ax-std v0.5.28 (/work/tgskits/os/arceos/ulib/axstd)
Compiling arceos-helloworld v0.1.0 (/work/tgskits/apps/arceos/helloworld)
Finished `release` profile [optimized] target(s) in 105m 17s
```

（后续若无特殊说明，也都是在同一机器上进行的测试，数据可以直接对比）

3.2 调优过程

3.2.1 初期调优

由于测试环境没有基准比较，不知道是哪个系统调用快或慢，初期的调优主要靠静态分析，我们优化了：

- 多核调度策略负载均衡调整；
- 内存管理锁的优化；
- 多核独立内核堆分配器；
- 用户引用内核页表与按需 tlb 刷新；
- ...

尽管单独测试发现这些优化为各子项带来了显著提升，但没有根本解决 buildstorm 显著偏慢的问题。

3.2.2 与 alpine linux 作对比

而后我们选择了运行 alpine linux 的 elf 镜像作为基线参照。我们在 alpine linux 上运行 buildstorm 脚本，发现耗时约为 7min，显著快于我们的内核。于是我们用 strace 跟踪基线，得到了下列系统调用耗时表：

% time	seconds	usecs/call	calls	errors	syscall
56.97	6809.271731	45315	150263	10611	futex
13.88	1658.668204	1587242	1045		ppoll
11.75	1404.142361	13069	107437	830	read
8.66	1035.674753	3362580	308	34	wait4
3.66	437.892711	437892711	1	1	rt_sigsuspend
3.61	431.497688	47944187	9	8	epoll_pwait
0.77	91.830055	199	459803		mprotect
0.15	17.899470	67	264428	197	lseek
0.09	11.280825	138	81733	81416	readlinkat
0.08	9.267939	111	82833	12506	statx
0.06	7.392435	163	45245	18957	openat
0.06	6.763408	153	44192		write
0.05	5.511626	474	11605		munmap
0.03	4.114190	217	18908		mmap
0.03	3.229722	105	30656	2	close
0.02	2.028941	212	9541		rt_sigprocmask
0.01	1.573025	836	1881		clone3
0.01	1.306759	7341	178		recvfrom
0.01	1.100431	3255	338	49	execve
0.01	1.098155	188	5829		sigaltstack
0.01	1.049308	648	1619		madvise
0.01	0.971214	143	6762		getdents64
0.01	0.955752	104	9107		fstat
0.01	0.750188	90	8278	2	fcntl
0.00	0.597107	105	5645		brk
0.00	0.559801	398	1406	289	unlinkat
0.00	0.514519	102	5008	1321	newfstatat
0.00	0.484567	2253	215		clone
0.00	0.483706	384	1258	589	mkdirat
0.00	0.445917	192	2311		set_robust_list
0.00	0.430780	150	2871		getcwd
0.00	0.428119	193	2217		sched_getaffinity
0.00	0.417472	197	2110		rseq
0.00	0.368413	127	2898		rt_sigaction
0.00	0.359986	176	2035		gettid
0.00	0.312694	124	2504	1928	ioctl
0.00	0.296963	781	380		sched_yield
0.00	0.249905	208	1201	4	prctl
0.00	0.160295	165	969		getrandom
0.00	0.155119	562	276		mremap
0.00	0.126822	182	694	523	faccessat
0.00	0.122402	425	288		renameat2
0.00	0.118726	171	693		dup3
---SUMMARY_TAIL---					

当然，我们的内核并不支持用户 trace 工具，所以我们挑出其中的部分调用，单独编写测例分别测试我们内核与 alpine linux 的耗时，发现 mm 和 fs 的部分调用性能差距较大。但大多高频调用耗时只有 $1x \sim 4x$ ，不足以解释 20 倍左右的性能差距。后续单独优化 mm 等也发现收益有限。

于是我们把目光放在：是否是一些系统调用不满足 linux 语义导致的？于是我们选择把图上的系统调用拿回到初赛 ltp 中进行测试，确实发现了一些语义错误。

3.2.3 带来显著提升的优化

通过测试对系统调用的占比以及耗时有了大概了解，接下来对这些系统调用进行调试纠错即可。需要说明的是，这里只列举了几个代表性的改动，不能把进步完全归因于它们：由于多个改动可能在同一次提交中，且部分中间态有错误无法完成测试，我们不能完全做到控制单一变量；许多这里没提及的无锁化/架构调整（详见第一章概述和项目仓库的 commit 记录）同样是性能提升的因素。

1. futex 的纠错与优化

过去我们在适配 ltp 的简单 futex 时，采用了以物理地址作键的信号量队列。而在支持了写时复制之后的多线程环境下，信号量的物理地址也可能发生改变（当触发了写时复制后）；另外旧实现的 futex 部分采用轮询，效率十分低下。因此更新了 futex 的键管理机制后，我们的运行时间缩短到了 25min：

```
Compiling arceos-networld v0.1.0 (/work/tyoskits/apps/arceos/networld)
Finished 'release' profile [optimized] target(s) in 24m 59s
```

2. exec 的调优

我们每次的 exec 都要重新访问磁盘块重新读取，但实际上可以将 exec 阅读的 elf 放入缓存块，从而让反

复执行的程序可以更快地加载。修改后时间缩短到了 21min:

```
warning: `someboot` (lib) generated 1 warning
Compiling axpoll v0.5.1 (/work/tgoskits/components/axpoll)
Compiling flatten_objects v0.2.4
Compiling ax-io v0.6.1 (/work/tgoskits/components/axio)
Compiling ax-mm v0.5.28 (/work/tgoskits/os/arceos/modules/axmm)
Compiling ax-sync v0.5.28 (/work/tgoskits/os/arceos/modules/axsync)
Compiling ax-api v0.7.4 (/work/tgoskits/os/arceos/api/arceos_api)
Compiling ax-std v0.5.28 (/work/tgoskits/os/arceos/ulib/axstd)
Compiling arceos-helloworld v0.1.0 (/work/tgoskits/apps/arceos/helloworld)
Finished `release` profile [optimized] target(s) in 21m 43s
Converting ELF to BIN format...
```

3. ppoll epoll 方面的调优

这些系统调用在初赛阶段保留了开发初期的轮询效果（开发初期未支持阻塞，队员甚至还没系统学习操作系统课程）。决赛阶段将轮询改成阻塞，进一步将编译时间压缩到了 16min 左右：

```
Finished `release` profile [optimized] target(s) in 16m 03s
Converting ELF to BIN format...

elf: /work/tgoskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld

bin: /work/tgoskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
/root/.rustup/toolchains/nightly-2026-05-28-riscv64gc-unknown-linux-gnu/lib/rustlib/riscv64gc-unknown-linux-gnu/bin/
llvm-objcopy --strip-all -O binary /work/tgoskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld /work/tgoskits/
target/riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
[axbuild] cargo build package=arceos-helloworld target=scripts/targets/std/pie/riscv64gc-unknown-linux-musl.json config=/work/
tgoskits/tmp/axbuild/config/arceos-helloworld/build-riscv64gc-unknown-none-elf.toml ... done (1002.88s)
[axbuild] cargo build elf=/work/tgoskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
[axbuild] cargo build artifact_dir=/work/tgoskits/target/riscv64gc-unknown-linux-musl/release
BUILDSTORM_COMPILE mode=multi ok=true elapsed_s=1018.63 cores=8 bytes=1681000 arch=riscv64
#### OS COMP TEST GROUP END buildstorm-glibc ####
```

4. 文件系统中目录项负缓存与路径解析优化

上文可知即使在 alpine linux 中，readlinkat 也有大量调用以及失败，所以为了避免每次查找需要访问磁盘，将指定文件项不存在的结果缓存到内存中，

减少 readlinkat 在外存（或块缓存）中重新遍历目录项的次数。

同时优化路径解析和 getdents，减少了不必要的临时 String（会带来堆分配）和冗余缓冲区拷贝。

优化后的 buildstorm 耗时降到了 14min:

```

Compiling memchr v2.8.2
warning: function `set_fdt_addr_phys_if_valid` is never used
--> platforms/someboot/src/fdt/mod.rs:32:15
32 | pub(crate) fn set_fdt_addr_phys_if_valid(fdt_addr: usize) -> bool {
    |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    = note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

Compiling ax-ctor-bare v0.4.8 (/work/tgoskits/components/ctor_bare/ctor_bare)
Compiling linux-raw-sys v0.12.1
Compiling bitmaps v3.2.1
Compiling axpoll v0.5.1 (/work/tgoskits/components/axpoll)
Compiling ax-io v0.6.1 (/work/tgoskits/components/axio)
warning: `someboot` (lib) generated 1 warning
Compiling flatten_objects v0.2.4
Compiling ax-mm v0.5.28 (/work/tgoskits/os/arceos/modules/axmm)
Compiling ax-sync v0.5.28 (/work/tgoskits/os/arceos/modules/axsync)
Compiling ax-api v0.7.4 (/work/tgoskits/os/arceos/api/arceos_api)
Compiling ax-std v0.5.28 (/work/tgoskits/os/arceos/ulib/axstd)
Compiling arceos-helloworld v0.1.0 (/work/tgoskits/apps/arceos/helloworld)
Finished `release` profile [optimized] target(s) in 14m 01s
Converting ELF to BIN format...

elf: /work/tgoskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld
bin: /work/tgoskits/target/riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
[K] hart[7] PID92 syscall 291 returned EFAULT
[K] hart[0] PID98 syscall 291 returned EFAULT
/root/.rustup/toolchains/nightly-2026-05-28-riscv64gc-unknown-linux-gnu/lib/rustlib/
riscv64gc-unknown-linux-gnu/bin/llvm-objcopy --strip-all -O binary /work/tgoskits/target/
riscv64gc-unknown-linux-musl/release/arceos-helloworld /work/tgoskits/target/
riscv64gc-unknown-linux-musl/release/arceos-helloworld.bin
[axbuild] cargo build package=arceos-helloworld target=scripts/targets/std/pie/
riscv64gc-unknown-linux-musl.json config=/work/tgoskits/tmp/axbuild/config/arceos-helloworld/
build-riscv64gc-unknown-none-elf.toml ... done (878.15s)
[axbuild] cargo build elf=/work/tgoskits/target/riscv64gc-unknown-linux-musl/release/
arceos-helloworld
[axbuild] cargo build artifact_dir=/work/tgoskits/target/riscv64gc-unknown-linux-musl/release
BUILDSTORM_COMPILE mode=multi ok=true elapsed_s=885.91 cores=8 bytes=1681000 arch=riscv64
#### OS COMP TEST GROUP END buildstorm-glibc ####

```

5. 调度优化

我们观察发现：调整无任务核心的 wfi 时间会带来一定程度的 futex 响应性变化和测试的时间变化。进一步分析：新唤醒的任务总是被放在本核队列，可能

很久才会被 wfi 核醒来后窃取，这会导致 futex 响应变慢；而如果不 wfi，改为 spin_loop() 又会导致空闲核数量多时高频争抢锁，并浪费 cpu 资源。由此改进了调度策略：任务唤醒时，优先选择空闲核插入，并发送 ipi 打断其休眠状态；为每核用原子变量维护任务调度状态，减少队列锁竞争。

修改后用单独测例测试 futex 平均响应速度，从 20us 左右进步到了 13us 左右，而 buildstorm 进一步来到了 12min 左右：

```
Compiling ax-std v0.5.28 (/work/tgoskits/os/arceos/ulib/axstd)
Compiling arceos-helloworld v0.1.0 (/work/tgoskits/apps/arceos/helloworld)
Finished `release` profile [optimized] target(s) in 12m 08s
Converting ELF to BIN format...
```

6. 龙芯 bug 修复

前面的内容主要以 riscv 为目标，这是因为龙芯存在收尾阶段异常卡死的问题。gdb 调试发现龙芯在 tlb refill 路径上出现问题。进一步分析发现，动态加载时不会提前建立页表：这可能导致 lddir 加载到空目录项，从而使 cpu 将物理页 0 作为下一级页表节点访问而不正确触发缺页异常。修复后通过了龙芯下的 buildstorm 编译，时间与 riscv 接近。

3.2.4 AI 使用说明

前文提到，我们编写单独测例来和 linux 对照测试

syscall 单项/类别的性能或语义。这是一项繁琐重复的工作，我们用 Agent 工具辅助完成测例编写与注入，并将测试结果转录为.md 文本便于后续参考。相关内容也被我们上传到了仓库内的 benchmarks 文件夹，并添加了 AI 声明。

我们用 LLM 辅助阅读了 linux 源码，由此学习借鉴 linux 部分系统调用语义或部分管理的优秀思想。

部分优化也在我们提出方案后由 LLM 快速实现早期版本，由人工进行复核测试+修改完善。我们始终坚持：AI 会不代表我们会，学到东西最重要。因此本项目由 AI 生成的代码均经过人工审核，并且我们还会刻意学习其中的优美实现。

基于上述三点，具体开发过程应该可以较为容易地通过决赛阶段的提交记录与日志等重现。